Capacidad de recuperación: de "hay respaldos" a "se puede restaurar"

> **Belona Cybersecurity** · julio–octubre 2026 Caso real: plataforma en producción de una firma legal (Cliente A).
> *English version:* `english/disaster-recovery-EN.md`

El problema

Una firma legal pequeña opera un sistema que guarda **expedientes de clientes en PDF** y los datos asociados. No había respaldos. Si alguien borraba algo por error, o un proceso lo corrompía, no existía forma de volver atrás.

Para esa firma eso no es un riesgo informático: es un **riesgo profesional**. Perder el expediente de un cliente no se arregla con disculpas.

Pero el problema real era más sutil que "faltan respaldos", y aparece en casi todas las empresas que sí los tienen:

> **Tener copias y poder recuperar no son lo mismo**, y casi nadie comprueba la segunda.

El criterio, antes de la herramienta

Estas decisiones son las mismas en Google Cloud, en AWS, en Azure o en un servidor propio. **La plataforma solo cambia los comandos.**

Las dos preguntas que definen todo

	Qué pregunta	Qué decide
RPO	¿cuántos datos puedo permitirme perder?	cada cuánto se copia
RTO	¿cuánto puedo estar caído?	cómo se restaura

Un despacho que recibe pocos documentos al día tolera un RPO de horas. Uno que factura por minuto, no. **Nadie puede responder esas dos preguntas por el cliente**: son decisiones de negocio disfrazadas de decisiones técnicas, y preguntarlas es la mitad del trabajo.

La regla 3-2-1

> **3** copias · en **2** medios distintos · con **1** fuera del sitio

La parte que se incumple siempre es la última. Una copia en la misma cuenta, la misma región o el mismo proveedor que el original **comparte su destino**: si se pierde el acceso a la cuenta —por un error de facturación, una credencial comprometida o una suspensión— se pierden las dos a la vez.

Lo que se implementó

Capa	Qué protege	Ventana
Recuperación a un punto en el tiempo	la base de datos completa	**7 días**
Copias diarias programadas	la base de datos	**14 días**
Versionado de archivos	los expedientes en PDF	**90 días**
Copia fuera del sitio	todo, en otra cuenta de almacenamiento	semanal
Código fuente	la aplicación	historial completo en Git

RPO ~0 dentro de la ventana de 7 días · **RTO ~minutos** por el tamaño de la base.

Las capas no son redundantes: cada una cubre un fallo que las otras no.

- La **recuperación a un punto en el tiempo** sirve cuando se sabe *cuándo* se rompió algo — "a las 10:15 estaba bien".
- Las **copias diarias** sirven cuando se descubre tarde.
- El **versionado** sirve para un solo archivo, sin restaurar nada más.
- La **copia externa** sirve cuando el problema es la cuenta entera.

Restaurar toda la base para recuperar un PDF es como demoler la casa para cambiar una bombilla. Por eso hay cuatro y no una.

El hallazgo: lo documentado no era lo real

Durante la verificación de octubre se ejecutaron las tres comprobaciones de estado. Dos confirmaron lo esperado. **La tercera no.**

```
Punto en el tiempo:  HABILITADO            ✅
Copias diarias:      retención 14 días     ✅
Versionado:          (sin configurar)      ❌
```

El versionado de archivos nunca se activó. La documentación lo declaraba implementado, y sobre él se apoyaba el procedimiento de "recuperar un documento borrado" — que por tanto **no habría funcionado**.

Tres meses de confianza sobre un control que no existía.

El matiz, que importa

La primera lectura fue *"un archivo borrado se perdió"*. **Era exagerada.**

Al pedir la configuración completa del almacenamiento —en vez de filtrarla— apareció una política de borrado suave de **7 días** que el proveedor activa por defecto. Había una red de seguridad que nadie había documentado.

El tamaño real del hallazgo:

	Antes	Después
Ventana de recuperación de un expediente	**7 días**	**90 días**

Sigue siendo un hallazgo —la documentación mentía y el procedimiento no funcionaba— pero no era pérdida total. **Decirlo con precisión importa más que decirlo con fuerza.**

Corregido

El versionado se activó y se añadió una regla de ciclo de vida que elimina las versiones antiguas a los 90 días. Sin esa regla, cada edición de un documento guardaría una copia **para siempre** y el almacenamiento crecería sin control.

```
versioning_enabled: true
lifecycle: borrar versiones no vigentes > 90 días
```

⚠️ **El versionado solo protege desde que se activa.** Lo borrado antes no vuelve.

Cómo se encontró, que es la parte reutilizable

El hallazgo no salió de una auditoría ni de una herramienta. Salió de **ir a ejecutar la prueba de restauración** y empezar por verificar que lo documentado existía.

Dos detalles del método que fueron decisivos:

1. Pedir la salida completa, no filtrada. La primera consulta usaba un filtro por nombre de campo y devolvía `null`. Se interpretó como *"está desactivado"*. El campo real tenía otro nombre: `null` significaba *"ese campo no existe"*, no *"el control está apagado"*.

La salida completa resolvió la ambigüedad **y además reveló la política de borrado suave** que nadie había mencionado — el dato que corrigió la severidad.

> Un filtro oculta exactamente lo que no se te ocurrió preguntar.

2. No confiar en un resultado negativo sin probar la herramienta. Antes, en este mismo proyecto, se verificó que unos documentos no contenían cierto texto con una herramienta que **no sabía leer ese formato**. No encontraba nada porque no podía, no porque no hubiera nada.

> Un "no encontré nada" solo vale si la herramienta encuentra algo que **sí** debería encontrar.

Lo que sigue pendiente

La prueba de restauración completa no se ha ejecutado. Los respaldos están configurados y verificados como existentes; que se puedan restaurar de punta a punta está **deducido, no comprobado**.

Es el pendiente principal de este trabajo y está declarado aquí a propósito:

> **Un respaldo que nunca se ha restaurado no es un respaldo verificado. Es un respaldo supuesto.**

El procedimiento está escrito —restaurar a una base temporal, validar, eliminar la temporal— y no toca producción. Pendiente de ejecutarse y registrarse.

Dos observaciones menores encontradas en la misma revisión:

- El almacenamiento usa listas de control de acceso por objeto en vez de permisos unificados. Dos sistemas de permisos conviviendo es más fácil de equivocar que uno.
- El bucket principal está en una sola región. Conviene confirmar que el de respaldos sí es multirregión, como declara la documentación — **después de hoy, esa afirmación no se da por buena sin comprobarla.**

Lo que un cliente recibe

1. Las dos preguntas respondidas **por él**: cuántos datos puede perder y cuánto puede estar caído.
2. Las capas implementadas, **verificadas con la salida del sistema**, no con la pantalla de configuración.
3. **La prueba de restauración ejecutada delante suyo**, o la fecha en que se hará.
4. El procedimiento escrito para que lo ejecute alguien que no estuvo en la implementación.

El punto 3 es el que casi nadie entrega. **Es también el único que demuestra que lo demás sirve.**

Anexo — la implementación en Google Cloud

Los comandos concretos de este caso. **El equivalente existe en AWS** (versionado de S3, AWS Backup, políticas de ciclo de vida) **y en Azure** (blob versioning, Azure Backup). Lo que no cambia entre plataformas es todo lo anterior.

```
# Verificar que cada capa existe de verdad
gcloud firestore databases describe --database="(default)" \
  --format="value(pointInTimeRecoveryEnablement)"
```

```
gcloud firestore backups schedules list --database="(default)"
gcloud storage buckets describe gs://<BUCKET> --format=json   # completa, sin filtrar

# Activar versionado + retención de versiones antiguas
gcloud storage buckets update gs://<BUCKET> --versioning
gcloud storage buckets update gs://<BUCKET> --lifecycle-file=ciclo.json

# Restaurar a una base NUEVA. Nunca encima de la original.
gcloud firestore backups list --format="table(name,snapshotTime,state)"
gcloud firestore databases restore \
  --source-backup="<BACKUP>" --destination-database="restore-tmp"

# Recuperar un archivo concreto desde una versión anterior
gcloud storage ls -a "gs://<BUCKET>/<RUTA>"          # incluye versiones borradas
gcloud storage cp "gs://<BUCKET>/<RUTA>#<GENERACION>" "gs://<BUCKET>/<RUTA>"
```

La regla que no depende de la plataforma: se restaura a un destino nuevo y se valida antes de tocar el original. Importar encima de producción para "ver si funciona" es cómo un ensayo de recuperación se convierte en el incidente.

Caso del portafolio técnico de Belona Cybersecurity. Trabajo real sobre una plataforma en producción, con autorización del propietario.
